

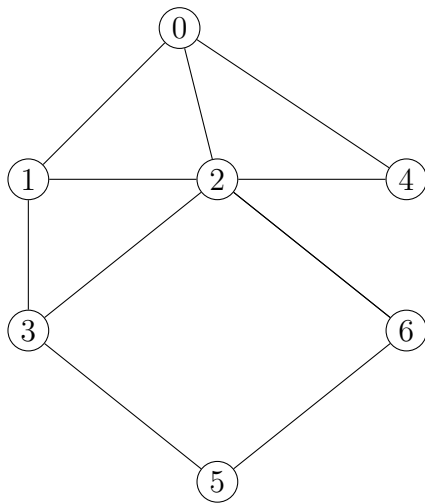
- TD 9, Partie 2 -

Parcours d'un graphe en profondeur : DFS (*Depth First Search*)

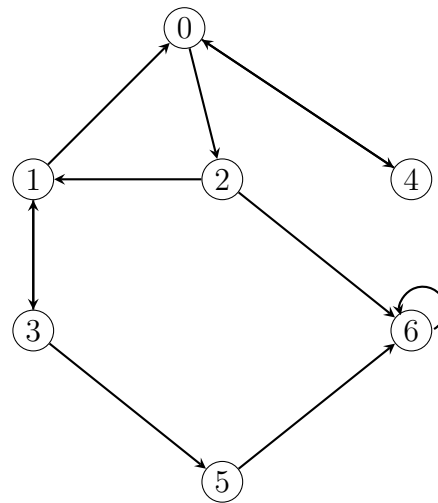
Exercice 1 :

1. On considère les deux graphes ci-dessous :

Graphe G_1 :



Graphe G_2 :



- (a) Déterminer « à la main » la liste des sommets visités à partir du sommet source s , pour $s = 5$, puis $s = 2$, puis pour $s = 1$, à l'aide de l'algorithme de Parcours en profondeur. A chaque étape de l'algorithme, parmi les voisins ou successeurs non encore visités, on choisira celui de plus petit numéro.

On pourra remplir un tableau de la forme :

Étape initiale	Access=[s]	Avisiter=[s]
Fin de la boucle while n° 1	Access=	Avisiter=
Fin de la boucle while n° 2	Access=	Avisiter=
Fin de la boucle while n° 3	Access=	Avisiter=
Fin de la boucle while n° 4	Access=	Avisiter=
Fin de la boucle while n° 5	Access=	Avisiter=
Fin de la boucle while n° 6	Access=	Avisiter=
Fin de la boucle while n° 7	Access=	Avisiter=

Dans chaque cas, tracer le « sous-graphe » obtenu : tracer l'arbre constitué des sommets donnés dans la liste **Access** et dont les arêtes ou arcs sont ceux parcourus dans le parcours DFS pour passer d'un sommet visité à un nouveau sommet à visiter.

- (b) Définir les deux graphes précédents sous la forme d'un dictionnaire, où pour chaque sommet s , la liste des voisins ou successeurs de s sera donnée par ordre croissant. Vérifier alors vos réponses à l'aide de la commande Python `nx.dfs_tree`. Pour cela, on pourra utiliser les commandes Python ci-dessous :

```
import matplotlib.pyplot as plt
import networkx as nx
D= # Définition du dictionnaire associé au graphe
G=nx.Graph(D) # ou nx.DiGraph(D)
s=5
Gdfs=nx.dfs_tree(G,source=s)
nx.draw(Gdfs,with_labels=True)
plt.show()
```

2. Vous pourrez vérifier la validité de vos fonctions en utilisant les exemples donnés précédemment (ou tout autre graphe), pour différents sommets sources, et comparer le résultat à celui obtenu par la commande Python `nx.dfs_tree`.

- (a) Compléter la fonction Python `parcoursDFS` ci-dessous, de paramètres G de type `dict` représentant un graphe (orienté ou non), et s représentant l'un des sommets de G , et renvoyant la liste des sommets accessibles depuis s via l'algorithme de parcours DFS.

```
def ParcoursDFS(G,s):
    Access=[s]
    Avisiter=[s]
    while Avisiter!=[]:
        u=Avisiter.pop()
        L=list(G[u])
        d=len(L)
        k=0
        while k<d:
            x=L[k]
            if not (x in Access):
                Avisiter.append(x)
                Access.append(x)
            k=k+1
        return(Access)
```

Quelles commandes devez-vous ajouter pour obtenir, lors de l'exécution de la fonction Python, un tableau analogue à celui rempli dans la question 1a ?

- (b) Ecrire une fonction en Python nommée `parcoursDFS2`, de paramètres M de type `array` représentant la matrice d'adjacences d'un graphe G d'ordre n (orienté ou non) de sommets nommés $0, 1, \dots, n-1$, et s représentant l'un des sommets de G , et renvoyant la liste des sommets accessibles depuis s par un parcours du graphe en profondeur.

3. Réécrire la fonction de la question 2a sous la forme d'un algorithme récursif.

Partant d'un sommet s d'un graphe G , le parcours du graphe en profondeur obtenu avec l'algorithme récursif est-il le même que celui obtenu avec l'algorithme itératif ?

Exercice 2 : Applications de l'algorithme DFS

1. Test de connexité d'un graphe non orienté

On rappelle que si G est un graphe non orienté,

G est connexe si, et seulement si, il existe un sommet s de G tel que tout sommet de G , distinct de s , est accessible à partir s .

Écrire une fonction Python `Est_connexe`, de paramètre `G` de type `dict` et représentant un graphe non orienté, utilisant la fonction `ParcoursDFS` de l'exercice 1, et qui renvoie '`Connexe`' si le graphe `G` est connexe, et '`Non Connexe`' sinon.

2. Arbre couvrant d'un graphe non orienté et connexe

(a) Soit G un graphe non orienté d'ordre n et connexe. Soit s un sommet de G .

On note :

- $L = [s_1, s_2, \dots, s_n]$ la liste des sommets obtenus par l'algorithme DFS, dans l'ordre de leur découverte, avec $s_1 = s$,
- C l'ensemble des arêtes du graphe G qui ont été parcourues lors du parcours en profondeur de G en partant du sommet s .

On définit ainsi un nouveau graphe : $G_s = (L, C)$.

Justifier que le graphe G_s est un arbre.

(b) Écrire une fonction en Python nommée `arbrecouvrant`, de paramètre `M` de type `array` représentant un graphe G non orienté et connexe et `s` représentant l'un des sommets de G , et qui renvoie la matrice d'adjacence de l'arbre G_s défini dans la question précédente. *On pourra adapter la fonction `ParcoursDFS` de l'exercice précédent, en y ajoutant une liste donnant, pour chaque sommet s de G , le sommet u qui a permis de découvrir le sommet s lors du parcours.*

3. On considère la fonction Python ci-dessous, utilisant la fonction `ParcoursDFS` de l'exercice 1 :

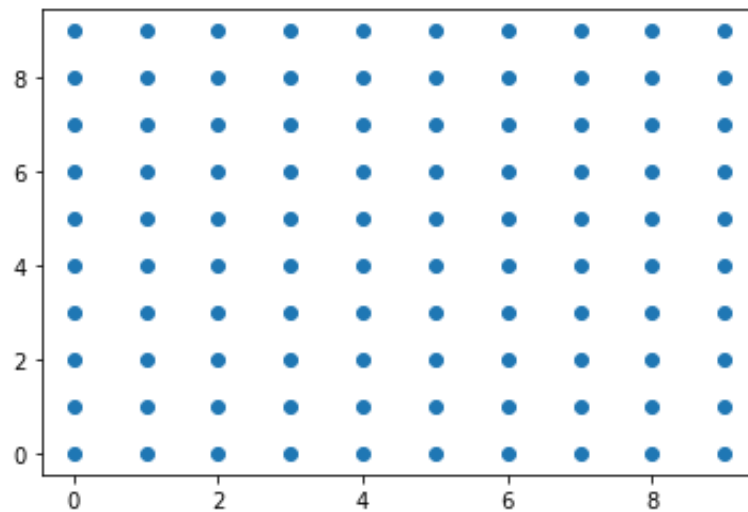
```
def Mystere(G):
    n=len(G)
    Atraiter=list(G.keys())
    C=0
    while Atraiter!=[]:
        C=C+1
        x=Atraiter.pop()
        Lx=ParcoursDFS(G,x)
        Lx.remove(x)
        for y in Lx:
            Ly=ParcoursDFS(G,y)
            if x in Ly:
                Atraiter.remove(y)
    return(C)
```

Expliquer ce que renvoie la fonction `Mystere` lorsque l'on exécute cette fonction, pour un paramètre `G` de type `dict` représentant un graphe orienté.

Exercice 3 : Comparaison des parcours en profondeur et en largeur

On dispose d'une grille $\mathcal{G}_n$ composée de $n \times n$ points, définis comme les points d'intersection de n lignes horizontales et de n lignes verticales.

Exemple de grille pour $n = 10$:



On munit le plan d'un repère orthonormé de sorte que les points de la grille $\mathcal{G}_n$ soient les points (x, y) de coordonnées $x \in \llbracket 0, n-1 \rrbracket$ et $y \in \llbracket 0, n-1 \rrbracket$.

Un pion se déplace sur la grille soit d'une unité sur la droite, soit d'une unité sur la gauche, soit d'une unité vers le haut, soit d'une unité vers le bas, lorsque ces déplacements sont possibles.

Autrement dit, si le pion se situe au point de coordonnées (x, y) , il peut se déplacer sur le point $(x', y') \in \llbracket 0, n-1 \rrbracket^2$ tel que $|x - x'| + |y - y'| = 1$.

On résume ces règles par un graphe G_n dont les sommets sont les coordonnées (x, y) des points de la grille $\mathcal{G}_n$, deux sommets (x, y) et (x', y') étant reliés par une arête si, et seulement si, $|x - x'| + |y - y'| = 1$.

1. Représenter graphiquement le graphe G_3 .
2. Écrire une fonction Python `Grille` de paramètre `n`, de type `int`, qui renvoie un dictionnaire, représentant le graphe défini ci-dessus
3. (a) On donne ci-après les deux fonctions Python `PP1` et `PP2`, de paramètres `n` de type `int`, et `depart` de type `tuple` (ou « couple ») de deux entiers représentant un point de la grille, et qui effectue le parcours en profondeur du graphe G_n en partant du sommet source `depart`, puis qui renvoie un couple `(liste_x, liste_y)` de deux listes donnant les abscisses et les ordonnées des sommets visités, dans l'ordre du parcours.
Pour chacune des deux fonctions, exécutée pour $n = 2$ et `depart`=(0,0), puis pour `depart`=(1,1), faites tourner l'algorithme « à la main » (c'est-à-dire sans utiliser l'ordinateur et Python) et donner le couple de listes obtenues à l'issue de l'algorithme. Vérifier ensuite vos résultats à l'aide de l'ordinateur.
Les deux résultats obtenus, pour les mêmes paramètres `n` et `depart`, en utilisant chacune des deux fonctions sont-ils identiques ? Expliquez pourquoi.

```

def PP1(n, depart):
    G=Grille(n)
    pile = [depart]

    marquage = {}
    for (x,y) in G.keys():
        marquage[(x,y)] = 'Blanc'

    (x,y)=depart
    marquage[(x,y)]= 'Gris'
    liste_x, liste_y = [x], [y]

    while len(pile) > 0:
        (x, y) = pile.pop()
        L=list(G[(x,y)])
        d=len(L)
        k=0
        while k<d:
            (a,b)=L[k]
            if marquage[(a,b)]=='Blanc':
                marquage[(a,b)]= 'Gris'
                pile.append((x,y))
                pile.append((a,b))
                liste_x.append(a)
                liste_y.append(b)
            k=d
        k=k+1
    return (liste_x, liste_y)

```

```

def PP2(n, depart):
    G=Grille(n)
    pile = [depart]

    marquage = {}
    for (x,y) in G.keys():
        marquage[(x,y)] = 'Blanc'

    liste_x, liste_y = [], []

    while len(pile) > 0:
        (x, y) = pile.pop()
        if marquage[(x,y)]=='Blanc':
            marquage[(x,y)] = 'Gris'
            liste_x.append(x)
            liste_y.append(y)
            for voisin in G[(x,y)]:
                pile.append(voisin)
    return (liste_x, liste_y)

```

- (b) Écrire une fonction Python PL de paramètres n de type `int`, et `depart` de type `tuple` (ou « couple ») de deux entiers représentant un point de la grille, qui effectue le parcours en largeur du graphe G_n en partant du sommet source `depart`, puis qui renvoie un couple `(liste_x, liste_y)` de deux listes donnant les abscisses et les ordonnées des sommets visités, dans l'ordre du parcours.

4. On importera le module Python `matplotlib.pyplot` sous la forme :

```
import matplotlib.pyplot as plt
```

Lorsque `liste_x` et `liste_y` sont deux listes de réels de même taille, notées respectivement $[x_0, x_1, \dots, x_k]$ et $[y_0, y_1, \dots, y_k]$, les commandes Python :

```

liste_c=[i for i in range(k+1)]
plt.scatter(liste_x, liste_y, c=liste_c)
plt.show()

```

trace dans une fenêtre graphique le nuage de points de coordonnées (x_i, y_i) , pour $i \in \llbracket 0, k \rrbracket$, avec un dégradé de couleurs.

Exécuter les lignes de commandes ci-dessous pour un exemple concret :

```

liste_c=[i for i in range(10)]
liste_x=[k for k in range(10)]
liste_y=[0]*10
for i in range(5):
    liste_y[2*i]=1
plt.scatter(liste_x, liste_y, c=liste_c)
plt.show()

```

- (a) Écrire une fonction `afficherPP1`, de paramètres `n` de type `int` et `depart` de type `tuple` d'entiers représentant un point de la grille $\mathcal{G}_n$, qui affiche la succession des points de la grille $\mathcal{G}_n$, obtenus par un parcours en profondeur du graphe $\mathcal{G}_n$ en partant du point de départ `depart`. On utilisera pour cela la fonction `PP1` de la question 3a et une liste de couleur permettant de repérer l'ordre de visite des points du graphe.
- (b) Écrire de même une fonction `afficherPL` de mêmes paramètres, et de même fonctionnalité, mais parcourant le graphe en largeur.
- (c) Comparer les résultats en faisant varier n et le point de départ.
- (d) Comparer vos graphiques avec ceux obtenus par votre voisin. Pourquoi vos représentations graphiques peuvent ne pas être identiques ?
